

Actividad 3 UT 3 - Restaurante



Índice

Enunciado	3
Explicación del código	5
Capturas de pantalla	43
Código completo	75
Acceso al zip del proyecto	104

Enunciado

Actividad 3 UT 3 - Restaurante

El objetivo de esta práctica es aprender a modelar y manejar relaciones complejas entre varias entidades usando Hibernate. Se trabajará con varias tablas relacionadas y se practicarán distintos tipos de relaciones (OneToMany,ManyToOne, ManyToMany)

Descripción

Se desea crear una aplicación que permita gestionar la información de un restaurante, incluyendo sus mesas, platos y comandas. La base de datos debe almacenar:

1. Mesas: Cada mesa tiene un número, capacidad (número máximo de comensales) y estado (libre/ocupada).
2. Platos: Cada plato tiene un nombre, descripción, precio y categoría (entrada, principal, postre, bebida).
3. Comandas: Una comanda se realiza en una mesa concreta y puede incluir varios platos.
4. Camareros: Cada camarero puede atender varias comandas, y cada comanda está atendida por un solo camarero.
5. Clientes: Un cliente puede tener varias comandas (por ejemplo, si viene varias veces al restaurante). Cada comanda pertenece a un cliente.

Entidades y relaciones

Mesa

id (PK)

numero

capacidad

estado

Relación: Una mesa puede tener muchas comandas (OneToMany).

Plato

id (PK)

nombre

descripcion

precio

categoria

Relación: Un plato puede estar en muchas comandas.

Comanda

id (PK)

fechaHora

Relación:

- Pertenece a una mesa (ManyToOne).
- Está atendida por un camarero (ManyToOne).
- Pertenece a un cliente (ManyToOne).

- Contiene varios platos (ManyToMany).

Camarero

id (PK)

nombre

dni

Relación: Un camarero puede atender muchas comandas (OneToMany).

Cliente

id (PK)

nombre

telefono

email

Relación: Un cliente puede tener muchas comandas (OneToMany).

Requisitos de la práctica

1. Crear las clases Java correspondientes a cada entidad con sus atributos y relaciones anotadas con Hibernate.

2. Mapear las relaciones correctamente, incluyendo:

- OneToMany

- ManyToOne

- ManyToMany con atributo adicional (cantidad).

3. Realizar operaciones básicas con Hibernate:

- Insertar mesas, platos, camareros y clientes.

- Crear comandas asociando platos con cantidad, cliente, camarero y mesa.

4. Realizar las siguientes consultas HQL

- Listar todas las comandas de una mesa concreta.

- Listar los platos de una categoría concreta.

- Obtener el total facturado por un camarero o un cliente en un rango de fechas.

- Consultar comandas de un cliente específico, mostrando la mesa y el camarero.

- Listar las mesas con estado "ocupada" junto con el número de comandas activas.

5. Implementar métodos para actualizar el estado de las mesas y eliminar comandas.

Opcional

Añadir categorías de platos y listar el total facturado por categoría.

Explicación del código

Main.java:

```
import modelo.*;
import dao.*;
import util.HibernateUtil;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        MesaDAO mesaDAO = new MesaDAO();
        PlatoDAO platoDAO = new PlatoDAO();
        CamareroDAO camareroDAO = new CamareroDAO();
        ClienteDAO clienteDAO = new ClienteDAO();
        ComandaDAO comandaDAO = new ComandaDAO();

        Scanner sc = new Scanner(System.in);
        int opt;

        do {
            System.out.println("\n--- MENU RESTAURANTE ---");
            System.out.println("1. Insertar datos de ejemplo");
            System.out.println("2. Crear comanda (con items)");
            System.out.println("3. Listar comandas de una mesa");
            System.out.println("4. Listar platos por categoria");
            System.out.println("5. Total facturado por camarero
(rango)");
            System.out.println("6. Total facturado por cliente
(rango)");
            System.out.println("7. Comandas de cliente (mostrar
mesa y camarero)");
            System.out.println("8. Listar mesas ocupadas con n°
comandas");
            System.out.println("9. Actualizar estado mesa");
            System.out.println("10. Eliminar comanda");
            System.out.println("11. Total facturado agrupado por
categoria");
            System.out.println("0. Salir");
            System.out.print("Opción: ");
            opt = Integer.parseInt(sc.nextLine());

            switch (opt) {
```

```

        case 1 -> {
            Mesa m1 = new Mesa(1, 4, "libre");
            Mesa m2 = new Mesa(2, 2, "ocupada");
            mesaDAO.guardar(m1);
            mesaDAO.guardar(m2);

            platoDAO.guardar(new Plato("Ensalada",
"Ensalada mixta", 6.5, "entrada"));
            platoDAO.guardar(new Plato("Solomillo",
"Solomillo a la brasa", 15.0, "principal"));
            platoDAO.guardar(new Plato("Tiramisu",
"Tiramisu de piña", 5.0, "postre"));
            platoDAO.guardar(new Plato("Cerveza", "Cerveza
Dorada", 2.5, "bebida"));

            camareroDAO.guardar(new Camarero("Laura",
"12345678Z"));
            clienteDAO.guardar(new Cliente("Carlos",
"600111222", "c@ejemplo.com"));

            System.out.println("Datos de ejemplo
insertados.");
        }

        case 2 -> {
            System.out.print("ID mesa: ");
            Long idMesa = Long.parseLong(sc.nextLine());
            System.out.print("ID camarero: ");
            Long idCam = Long.parseLong(sc.nextLine());
            System.out.print("ID cliente: ");
            Long idCli = Long.parseLong(sc.nextLine());

            Mesa mesa = mesaDAO.obtener(idMesa);
            Camarero camarero = camareroDAO.obtener(idCam);
            Cliente cliente = clienteDAO.obtener(idCli);

            if (mesa == null || camarero == null || cliente
== null) {
                System.out.println("IDs inválidos. No se
puede crear la comanda.");
                break;
            }

            // La mesa pasa a "ocupada" al crear una
comanda

            mesa.setEstado("ocupada");
            mesaDAO.actualizar(mesa);

```

```

        Comanda comanda = new
Comanda(LocalDate.now(), mesa, camarero, cliente);

        System.out.print("¿Cuántos platos en la
comanda? ");

        int n = Integer.parseInt(sc.nextLine());

        for (int i = 0; i < n; i++) {
            System.out.print("ID plato: ");
            Long idPlato =
Long.parseLong(sc.nextLine());
            System.out.print("Cantidad: ");
            int cantidad =
Integer.parseInt(sc.nextLine());

            Plato plato = platoDAO.obtener(idPlato);

            if (plato != null) {
                ComandaPlato cp = new ComandaPlato();
                cp.setPlato(plato);
                cp.setCantidad(cantidad);

                comanda.addItem(cp);
            } else {
                System.out.println("Plato no
encontrado: " + idPlato);
            }
        }

        comandaDAO.crearComanda(comanda);

        System.out.println("Comanda creada con
éxito:");

        System.out.println(comanda);
    }

    case 3 -> {
        System.out.print("ID mesa: ");
        Long id = Long.parseLong(sc.nextLine());
        List<Comanda> lista =
comandaDAO.comandasPorMesa(id);

        lista.forEach(c -> {
            System.out.println(c);
            c.getItems().forEach(it ->
                System.out.println("    - " +
it.getPlato().getNombre() + " x" + it.getCantidad())
            );
        });
    }
}

```

```

        });
    }

    case 4 -> {
        System.out.print("Categoria
(entrada/principal/postre/bebida): ");
        String cat = sc.nextLine();

        platoDAO.listarPorCategoria(cat).forEach(System.out::println);
    }

    case 5 -> {
        System.out.print("ID camarero: ");
        Long idC = Long.parseLong(sc.nextLine());
        System.out.print("Desde (yyyy-MM-ddTHH:mm)
(ejemplo 2025-12-01T14:30): ");
        LocalDateTime desde =
LocalDateTime.parse(sc.nextLine());
        System.out.print("Hasta (yyyy-MM-ddTHH:mm): ");
        LocalDateTime hasta =
LocalDateTime.parse(sc.nextLine());

        Double total =
comandaDAO.totalFacturadoPorCamarero(idC, desde, hasta);
        System.out.println("Facturado: " + (total ==
null ? 0 : total) + " €");
    }

    case 6 -> {
        System.out.print("ID cliente: ");
        Long idCl = Long.parseLong(sc.nextLine());
        System.out.print("Desde (yyyy-MM-ddTHH:mm)
(ejemplo 2025-12-01T14:30): ");
        LocalDateTime desde =
LocalDateTime.parse(sc.nextLine());
        System.out.print("Hasta (yyyy-MM-ddTHH:mm): ");
        LocalDateTime hasta =
LocalDateTime.parse(sc.nextLine());

        Double total =
comandaDAO.totalFacturadoPorCliente(idCl, desde, hasta);

        System.out.println("Total facturado por
cliente: " + (total == null ? 0 : total) + " €");
    }

    case 7 -> {
        System.out.print("ID cliente: ");

```



```

        Long idCl = Long.parseLong(sc.nextLine());
        List<Comanda> cs =
comandaDAO.comandasPorCliente(idCl);

        cs.forEach(c ->
            System.out.println(
                c +
                    " | Mesa: " +
(c.getMesa() != null ? c.getMesa().getNumero() : "-") +
                    " | Camarero: " +
(c.getCamarero() != null ? c.getCamarero().getNombre() : "-")
                )
            );
        }

        case 8 -> {
            List<Object[]> res =
mesaDAO.mesasOcupadasConCuentaComandas();
            res.forEach(row -> {
                Mesa m = (Mesa) row[0];
                Long contador = (Long) row[1];
                System.out.println(m + " -> nº comandas: "
+ contador);
            });
        }

        case 9 -> {
            System.out.print("ID mesa: ");
            Long idM = Long.parseLong(sc.nextLine());
            System.out.print("Nuevo estado (libre/ocupada):
");

            String est = sc.nextLine();
            mesaDAO.actualizarEstado(idM, est);
            System.out.println("Estado actualizado.");
        }

        case 10 -> {
            System.out.print("ID comanda: ");
            Long idCo = Long.parseLong(sc.nextLine());
            comandaDAO.eliminarComanda(idCo);
            System.out.println("Comanda eliminada (si
existía).");
        }

        case 11 -> {
            List<Object[]> lista =
comandaDAO.totalFacturadoPorCategoria();

```

```

        System.out.println("\n--- Total facturado por
categoría ---");
        for (Object[] row : lista) {
            String categoria = (String) row[0];
            Double total = (Double) row[1];
            System.out.println(categoria + ": " +
(total == null ? 0 : total) + " €");
        }
    }

    } while (opt != 0);

    sc.close();
    HibernateUtil.shutdown();
}
}

```

El código Main.java representa el punto de entrada de una pequeña aplicación de consola que gestiona un restaurante utilizando Hibernate para acceder a la base de datos. Al inicio del main se crean las instancias de los distintos DAO: MesaDAO, PlatoDAO, CamareroDAO, ClienteDAO y ComandaDAO. Estos objetos son los responsables de realizar las operaciones CRUD contra la base de datos para cada entidad. Después se prepara un Scanner para leer las opciones del usuario y comienza un bucle do-while que muestra un menú con diferentes opciones del sistema, permitiendo realizar distintas acciones según lo que el usuario escriba.

La primera opción del menú inserta datos de ejemplo en la base de datos. Se crean dos mesas, varios platos y también un camarero y un cliente. Todo se guarda mediante los métodos guardar de cada DAO. Esto sirve para tener información inicial con la que poder probar el resto de opciones.

La segunda opción permite crear una nueva comanda. El programa pide los identificadores de mesa, camarero y cliente, los busca en la base de datos y comprueba que existan. Si alguno de ellos no está registrado, la comanda no se crea. Si todo está correcto, la mesa cambia a estado ocupada y se actualiza en la base de datos. Luego se crea un objeto Comanda, al que se irán añadiendo los distintos platos que el usuario indique. Para cada plato se solicita su id y la cantidad. Si el plato existe se crea un ComandaPlato y se añade a la comanda. Al finalizar, la comanda se guarda usando el método crearComanda, que persiste tanto la comanda como sus items. El programa muestra por pantalla el resultado.

La tercera opción lista todas las comandas asociadas a una mesa concreta. Se pide el id de la mesa y se consultan las comandas de esa mesa mediante el método comandasPorMesa del DAO. Luego se imprime cada comanda junto con sus items, mostrando también el nombre del plato y la cantidad. Como en la consulta se hace un fetch join, los items ya vienen cargados y no provocan errores al mostrarlos.

La cuarta opción sirve para listar platos por categoría. El usuario escribe la categoría y se imprimen todos los platos de ese tipo recuperados desde PlatoDAO.

La quinta opción calcula el total facturado por un camarero en un rango de fechas. Se pide el id del camarero y dos fechas en formato LocalDateTime. Con esos datos se llama al método totalFacturadoPorCamarero, que devuelve la suma del importe de los platos atendidos por ese camarero en ese periodo. Si el resultado es null se muestra como cero.

La sexta opción funciona igual pero para un cliente. Se pide su id y el rango de fechas y se calcula cuánto ha gastado dicho cliente en ese intervalo.

La séptima opción consulta todas las comandas realizadas por un cliente concreto. Además de mostrar la información básica de la comanda, también muestra la mesa asociada y el nombre del camarero que la atendió, lo cual ayuda a ver mejor el contexto de cada pedido.

La octava opción muestra las mesas que están ocupadas junto con el número de comandas asociadas a cada una. El método mesasOcupadasConCuentaComandas devuelve una lista de objetos donde cada uno contiene una mesa y un contador. Después se imprime cada mesa y su número de comandas.

La novena opción permite cambiar el estado de una mesa. Se pide el id y el nuevo estado (libre u ocupada) y se llama al método actualizarEstado, que realiza la modificación en la base de datos.

La décima opción elimina una comanda según su id. El método eliminarComanda busca la comanda y la elimina si existe. Después se muestra un mensaje informando de que se ha eliminado, si estaba registrada.

La opción once muestra el total facturado agrupado por categoría de plato. El método totalFacturadoPorCategoria devuelve una lista de filas donde la primera columna es la categoría del plato y la segunda la suma total facturada. El programa recorre cada fila y muestra la categoría junto con su importe acumulado.

El menú se repite hasta que el usuario introduce cero. Al finalizar se cierra el scanner y se invoca HibernateUtil.shutdown para cerrar la SessionFactory y liberar recursos antes de que el programa termine.

modelo/Camarero.java:

```
package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;
```

```

@Entity
@Table(name = "camareros")
public class Camarero {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String dni;

    @OneToMany(mappedBy = "camarero", cascade = CascadeType.ALL)
    private Set<Comanda> comandas = new HashSet<>();

    public Camarero() {}
    public Camarero(String nombre, String dni) { this.nombre =
nombre; this.dni = dni; }

    // getters y setters
    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDni() { return dni; }
    public void setDni(String dni) { this.dni = dni; }
    public Set<Comanda> getComandas() { return comandas; }

    @Override
    public String toString() { return "Camarero{id=" + id + ",
nombre='" + nombre + "', dni='" + dni + "'}"; }
}

```

El código de la clase Camarero representa a un camarero dentro del modelo de datos del restaurante. Está anotado como entidad de JPA (Java Persistence API), lo que significa que Hibernate lo va a mapear directamente a una tabla real en la base de datos, en este caso la tabla se llama camareros. Cada camarero tiene un id que se genera de forma automática con una estrategia de identidad, así que no hace falta asignarlo manualmente. También tiene dos campos básicos: el nombre y el dni, que almacenan la información principal del trabajador.

La clase además define una relación uno a muchos con la entidad Comanda. Esto significa que un camarero puede estar asociado a varias comandas. La anotación mappedBy indica que la relación se controla desde el lado de Comanda, concretamente desde su atributo camarero, y el cascade all hace que si se guarda un camarero nuevo con comandas asociadas, Hibernate se encargue también de persistir esas comandas sin necesidad de llamadas extra. Para evitar problemas de inicialización se usa un HashSet como colección por defecto.

El constructor vacío es obligatorio para que Hibernate pueda crear objetos internamente. También hay un constructor que permite crear un camarero pasando directamente el nombre y el dni. A continuación se incluyen los métodos getter y setter habituales, que sirven para acceder y modificar las propiedades. Finalmente se sobrescribe el método toString, que devuelve una representación sencilla del camarero mostrando su id, su nombre y su dni, lo cual resulta útil para depurar y ver información por consola durante las pruebas.

modelo/Ciente.java:

```
package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "clientes")
public class Ciente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String telefono;
    private String email;

    @OneToMany(mappedBy = "cliente", cascade = CascadeType.ALL)
    private Set<Comanda> comandas = new HashSet<>();

    public Ciente() {}
    public Ciente(String nombre, String telefono, String email) {
        this.nombre = nombre; this.telefono = telefono; this.email
= email;
    }

    // getters y setters
    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTelefono() { return telefono; }
    public void setTelefono(String telefono) { this.telefono =
telefono; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
```

```

public Set<Comanda> getComandas() { return comandas; }

@Override
public String toString() { return "Cliente{id=" + id + ",
nombre=" + nombre + "'}"; }
}

```

La clase Cliente es una entidad del modelo que representa a un cliente del restaurante dentro de la base de datos. Está marcada con las anotaciones de JPA para que Hibernate pueda mapearla a la tabla clientes. Igual que ocurre con otras entidades, tiene un id que se genera automáticamente mediante la estrategia identity, lo que hace que la base de datos asigne el valor sin que tengamos que preocuparnos.

El cliente guarda algunos datos básicos: el nombre, el teléfono y el email, que son simplemente cadenas normales. Además mantiene una relación uno a muchos con la entidad Comanda. Esto quiere decir que un mismo cliente puede haber hecho varias comandas a lo largo del tiempo. La relación está definida con mappedBy, indicando que el dueño de la relación es el atributo cliente dentro de la clase Comanda, y con cascade all para que al guardar un cliente nuevo junto con sus comandas, estas también se guarden automáticamente. Para almacenar las comandas se usa un HashSet, que evita duplicados y funciona bien como colección por defecto.

La clase incluye un constructor vacío, necesario para que Hibernate pueda crear instancias internas sin problemas, y otro constructor que permite inicializar directamente los datos principales del cliente. Más abajo se encuentran los getters y setters habituales, que sirven para leer y modificar las propiedades desde otras partes del código. Por último, el método toString devuelve una representación simplificada del cliente, mostrando su id y su nombre, lo cual es útil para mostrar información en consola durante las pruebas o la depuración.

modelo/Mesa.java:

```

package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "mesas")
public class Mesa {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```

@Column(nullable = false, unique = true)
private int numero;

private int capacidad;

// "libre" / "ocupada"
private String estado;

@OneToMany(mappedBy = "mesa", cascade = CascadeType.ALL,
orphanRemoval = true)
private Set<Comanda> comandas = new HashSet<>();

public Mesa() {}
public Mesa(int numero, int capacidad, String estado) {
    this.numero = numero;
    this.capacidad = capacidad;
    this.estado = estado;
}

// getters y setters
public Long getId() { return id; }
public int getNumero() { return numero; }
public void setNumero(int numero) { this.numero = numero; }
public int getCapacidad() { return capacidad; }
public void setCapacidad(int capacidad) { this.capacidad =
capacidad; }
public String getEstado() { return estado; }
public void setEstado(String estado) { this.estado = estado; }
public Set<Comanda> getComandas() { return comandas; }

@Override
public String toString() {
    return "Mesa{id=" + id + ", numero=" + numero + ",
capacidad=" + capacidad + ", estado=" + estado + "}";
}
}

```

La clase Mesa representa una mesa del restaurante dentro del sistema y está mapeada a la tabla llamadas mesas mediante las anotaciones de JPA. Cada mesa tiene un id generado automáticamente por la base de datos usando la estrategia identity. Además de eso, se guarda el número de mesa, que es obligatorio y único, de forma que no puede haber dos mesas con el mismo número. También tiene la capacidad, que indica cuantas personas caben, y un estado que normalmente se usa para marcar si está libre u ocupada.

La mesa mantiene una relación uno a muchos con las comandas. Eso significa que sobre una misma mesa se pueden hacer varias comandas distintas a lo largo del día. La colección de comandas está configurada con `orphanRemoval`, así que si se elimina una comanda de la lista, Hibernate sabe que también tiene que eliminarla de la base de datos. Se usa un `HashSet` para almacenar las comandas porque evita duplicados y es adecuado para este tipo de relaciones.

La clase incluye un constructor vacío, que Hibernate necesita para poder crear objetos internamente, y un constructor más completo que permite crear una mesa indicando su número, capacidad y estado. Los getters y setters sirven para acceder a los distintos campos. En el `toString` se devuelve una descripción de la mesa con sus datos más importantes, lo que suele resultar útil cuando se muestran los objetos por consola mientras probamos que todo funciona.

modelo/Plato.java:

```
package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "platos")
public class Plato {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String descripcion;
    private double precio;

    // entrada, principal, postre, bebida
    private String categoria;

    // relación inversa con ComandaPlato
    @OneToMany(mappedBy = "plato", cascade = CascadeType.ALL,
orphanRemoval = true)
    private Set<ComandaPlato> items = new HashSet<>();

    public Plato() {}
    public Plato(String nombre, String descripcion, double precio,
String categoria) {
```



```

        this.nombre = nombre; this.descripcion = descripcion;
this.precio = precio; this.categoria = categoria;
    }

    // getters y setters
    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) {
this.descripcion = descripcion; }
    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }
    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria =
categoria; }
    public Set<ComandaPlato> getItems() { return items; }

    @Override
    public String toString() {
        return "Plato{id=" + id + ", nombre='" + nombre + "',
precio=" + precio + ", categoria=" + categoria + "}";
    }
}

```

La clase Plato representa uno de los platos del restaurante dentro del sistema. Está marcada como una entidad de JPA y se guarda en la tabla platos. Cada plato tiene un id que se genera automáticamente gracias a la estrategia identity, así que no hace falta asignarlo a mano.

Además del identificador, el plato tiene un nombre, una descripción más o menos corta, el precio y una categoría que suele ser algo como entrada, principal, postre o bebida. Estos campos describen lo necesario para poder mostrar cada plato y usarlo dentro de una comanda.

La relación importante de la clase es el conjunto de ComandaPlato, que es la tabla intermedia usada para relacionar comandas con platos y además guardar cuántas unidades se pidieron. Aquí se usa una relación uno a muchos porque un plato puede aparecer en muchas comandas distintas. El mappedBy indica que la clave real está en la otra clase. También se usa orphanRemoval, así que si un ítem deja de estar asociado al plato desde la colección, se borrará automáticamente en la base de datos.

La clase tiene el constructor vacío necesario para Hibernate y otro que permite crear un plato nuevo indicando sus datos principales. Los getters y setters dan acceso a los atributos. Finalmente, el método toString devuelve una representación sencilla del plato, mostrando su id, nombre, precio y categoría, lo que viene bien para depurar o mostrar información mientras se prueba el proyecto.

modelo/Comanda.java:

```
package modelo;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "comandas")
public class Comanda {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // fecha y hora de la comanda
    private LocalDateTime fechaHora;

    @ManyToOne
    @JoinColumn(name = "mesa_id")
    private Mesa mesa;

    @ManyToOne
    @JoinColumn(name = "camarero_id")
    private Camarero camarero;

    @ManyToOne
    @JoinColumn(name = "cliente_id")
    private Cliente cliente;

    // items (plato + cantidad) - entidad intermedia
    @OneToMany(mappedBy = "comanda", cascade = CascadeType.ALL,
orphanRemoval = true)
    private Set<ComandaPlato> items = new HashSet<>();

    public Comanda() {}
    public Comanda(LocalDateTime fechaHora, Mesa mesa, Camarero
camarero, Cliente cliente) {
        this.fechaHora = fechaHora; this.mesa = mesa; this.camarero
= camarero; this.cliente = cliente;
    }

    // helpers para gestionar la relación bi-direccional
    public void addItem(ComandaPlato item) {
        items.add(item);
        item.setComanda(this);
    }
}
```

```

    }

    public void removeItem(ComandaPlato item) {
        items.remove(item);
        item.setComanda(null);
    }

    // getters y setters
    public Long getId() { return id; }
    public LocalDateTime getFechaHora() { return fechaHora; }
    public void setFechaHora(LocalDateTime fechaHora) {
this.fechaHora = fechaHora; }
    public Mesa getMesa() { return mesa; }
    public void setMesa(Mesa mesa) { this.mesa = mesa; }
    public Camarero getCamarero() { return camarero; }
    public void setCamarero(Camarero camarero) { this.camarero =
camarero; }
    public Cliente getCliente() { return cliente; }
    public void setCliente(Cliente cliente) { this.cliente =
cliente; }
    public Set<ComandaPlato> getItems() { return items; }

    @Override
    public String toString() {
        return "Comanda{id=" + id + ", fechaHora=" + fechaHora + ",
mesa=" + (mesa != null ? mesa.getNumero() : null)
            + ", camarero=" + (camarero != null ?
camarero.getNombre() : null) + ", cliente=" + (cliente != null ?
cliente.getNombre() : null) + "}";
    }
}

```

La clase Comanda representa una comanda registrada en el sistema, es decir, el conjunto de platos que un cliente ha pedido en una mesa en un momento concreto. Está marcada como entidad con la anotación `@Entity` y asociada a la tabla comandas mediante `@Table`. Lo primero que aparece es el campo `id`, que se genera automáticamente con una estrategia de identidad. Después se guarda la fecha y hora en la que se creó la comanda usando un `LocalDateTime`, lo cual permite registrar el momento exacto del pedido.

La comanda está relacionada con tres entidades distintas: una mesa, un camarero y un cliente. Todas estas relaciones son de muchos a uno, porque muchas comandas pueden pertenecer a una misma mesa, o haber sido atendidas por el mismo camarero, o haber sido hechas por el mismo cliente. Por eso se usa `@ManyToOne` en los tres casos, junto con `@JoinColumn` para indicar la columna de la relación en la base de datos.

Luego aparece el conjunto de `items`, que no son platos directamente sino la entidad intermedia `ComandaPlato`. Esta estructura sirve para poder guardar no solo qué plato se pidió sino también cuántas unidades de cada uno. La relación es de uno a muchos con

respecto a Comanda, y se controla desde el lado de la clase ComandaPlato, por eso se usa mappedBy. También se aplican cascadas completas y eliminación de huérfanos para que los items se borren automáticamente cuando dejen de pertenecer a la comanda.

La clase tiene un constructor vacío y otro que permite crear una comanda inicializando la fecha, la mesa, el camarero y el cliente. Además, incorpora dos métodos auxiliares, addItem y removeItem, que se encargan de mantener la relación bidireccional entre la comanda y sus items. Estos métodos añaden o quitan el ComandaPlato de la colección y, al mismo tiempo, ajustan la referencia inversa en el propio item para evitar inconsistencias en la sesión de Hibernate.

Al final aparecen los getters y setters habituales para permitir acceder y modificar los datos básicos, y un método toString que construye una representación de la comanda mostrando su id, la fecha y algunos datos resumidos de la mesa, el camarero y el cliente, siempre comprobando antes que no sean nulos para evitar errores.

modelo/ComandaPlato.java:

```
package modelo;

import jakarta.persistence.*;

@Entity
@Table(name = "comanda_plato")
public class ComandaPlato {

    @EmbeddedId
    private ComandaPlatoId id = new ComandaPlatoId();

    @ManyToOne
    @MapsId("comandaId")
    @JoinColumn(name = "comanda_id")
    private Comanda comanda;

    @ManyToOne
    @MapsId("platoId")
    @JoinColumn(name = "plato_id")
    private Plato plato;

    private int cantidad;

    public ComandaPlato() {}
    public ComandaPlato(Comanda comanda, Plato plato, int cantidad) {
        this.comanda = comanda; this.plato = plato; this.cantidad = cantidad;
        if (comanda != null && plato != null) {
```

```

        this.id = new ComandaPlatoId(comanda.getId(),
plato.getId());
    }
}

// getters y setters
public ComandaPlatoId getId() { return id; }
public Comanda getComanda() { return comanda; }
public void setComanda(Comanda comanda) { this.comanda =
comanda; }
public Plato getPlato() { return plato; }
public void setPlato(Plato plato) { this.plato = plato; }
public int getCantidad() { return cantidad; }
public void setCantidad(int cantidad) { this.cantidad =
cantidad; }

@Override
public String toString() {
    return "ComandaPlato{comanda=" + (comanda != null ?
comanda.getId() : null) +
        ", plato=" + (plato != null ? plato.getNombre() :
null) + ", cantidad=" + cantidad + "}";
}
}

```

La clase ComandaPlato es la que se encarga de representar la relación entre una comanda y un plato concreto, junto con la cantidad pedida de ese plato. No es una entidad cualquiera, sino una entidad intermedia que une dos tablas principales y permite añadir información adicional, en este caso el número de unidades solicitadas. Esta marcada como entidad con `@Entity` y asociada a la tabla `comanda_plato`.

El identificador de esta clase no es un campo simple como un Long, sino un id compuesto. Por eso se usa `@EmbeddedId`, que encapsula dentro de un objeto ComandaPlatoId los dos valores que forman la clave: el id de la comanda y el id del plato. Esta estructura es bastante típica cuando se trabaja con relaciones muchos a muchos que necesitan atributos extra.

Luego aparecen dos relaciones ManyToOne, una con Comanda y otra con Plato. Ambas llevan `@MapsId` porque cada una de ellas corresponde a una parte de la clave compuesta, es decir, el id de la comanda por un lado y el id del plato por otro. También se añade `@JoinColumn` para indicar las columnas físicas que se van a usar para unir.

El campo cantidad es simplemente un entero que indica cuántas unidades de ese plato se pidieron en la comanda. La clase tiene un constructor vacío y otro que permite crear la relación inicializando la comanda, el plato y la cantidad. En este segundo constructor, si los dos objetos vienen distintos de null, se recalcula el id compuesto manualmente tomando los ids reales tanto de la comanda como del plato.

Después aparecen los getters y setters típicos, que permiten acceder y modificar cualquiera de los campos, incluyendo el id embebido. Finalmente, el método toString construye una representación textual bastante sencilla en la que aparece el id de la comanda, el nombre del plato y la cantidad, siempre con cuidado de no provocar un null pointer si alguno de los objetos no está inicializado.

modelo/ComandaPlatoId.java:

```
package modelo;

import jakarta.persistence.*;
import java.io.Serializable;
import java.util.Objects;

@Embeddable
public class ComandaPlatoId implements Serializable {
    private Long comandaId;
    private Long platoId;

    public ComandaPlatoId() {}
    public ComandaPlatoId(Long comandaId, Long platoId) {
        this.comandaId = comandaId; this.platoId = platoId; }

    // getters / setters
    public Long getComandaId() { return comandaId; }
    public void setComandaId(Long comandaId) { this.comandaId =
comandaId; }
    public Long getPlatoId() { return platoId; }
    public void setPlatoId(Long platoId) { this.platoId = platoId;
}

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof ComandaPlatoId)) return false;
        ComandaPlatoId that = (ComandaPlatoId) o;
        return Objects.equals(comandaId, that.comandaId) &&
Objects.equals(platoId, that.platoId);
    }

    @Override
    public int hashCode() { return Objects.hash(comandaId,
platoId); }
}
```

La clase ComandaPlatoId sirve para representar la clave primaria compuesta de la tabla intermedia que relaciona comandas con platos. Está marcada con la anotación @Embeddable porque no es una entidad independiente, sino un objeto que se incrusta dentro de otra clase, en este caso dentro de ComandaPlato. Implementa Serializable, que es un requisito obligatorio en Hibernate cuando se trabaja con claves compuestas, ya que estas deben poder serializarse sin problemas.

Dentro de esta clase se guardan dos campos simples: comandaId y platoId, que representan los identificadores de cada una de las entidades principales. El constructor vacío es necesario para que Hibernate pueda crear instancias internamente, y luego hay otro constructor que permite inicializar directamente ambos valores. También aparecen los métodos getter y setter habituales para acceder o modificar cada propiedad cuando haga falta.

Después vienen los métodos equals y hashCode, que son fundamentales en una clase usada como clave compuesta. El equals compara si dos objetos ComandaPlatoId representan exactamente la misma pareja de id de comanda y de plato, mientras que hashCode genera un valor hash consistente con esa comparación. Esto evita problemas al manejar objetos de este tipo en colecciones o cuando Hibernate los usa internamente para identificar registros. En resumen, esta clase actúa como contenedor de la clave de la relación entre comanda y plato, y proporciona lo necesario para que Hibernate pueda gestionar esa relación sin errores.

dao/ClienteDAO.java:

```
package dao;

import modelo.Cliente;
import modelo.Comanda;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.time.LocalDateTime;
import java.util.List;

public class ClienteDAO {

    public void guardar(Cliente c) {
        Transaction tx = null;
```

```

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Cliente obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Cliente.class, id);
        }
    }

    public List<Cliente> listarTodos() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Cliente",
Cliente.class).list();
        }
    }

    public void actualizar(Cliente c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Cliente c = s.find(Cliente.class, id);

```



```

        if (c == null) return;

        tx = s.beginTransaction();
        s.remove(c);
        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    }
}

public List<Comanda> listarComandasDeCliente(Long idCliente) {
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        String hql = "FROM Comanda c WHERE c.cliente.id =
:idCli";
        return s.createQuery(hql, Comanda.class)
            .setParameter("idCli", idCliente)
            .list();
    }
}
}

```

El código de ClienteDAO define una clase encargada de manejar todas las operaciones básicas relacionadas con la entidad Cliente dentro de Hibernate. Es una clase típica de acceso a datos que abre sesiones, inicia transacciones cuando hace falta y ejecuta las operaciones más comunes. En el método guardar se recibe un objeto Cliente y se persiste en la base de datos dentro de una transacción; si ocurre algún problema se hace un rollback para evitar que quede algo a medias. El método obtener simplemente abre una sesión y usa `s.find` para devolver el cliente cuyo id coincida con el que se pasa por parámetro.

El método `listarTodos` devuelve una lista completa de clientes usando una consulta HQL muy sencilla que pide todos los registros de la tabla. Para actualizar un cliente se utiliza `merge` dentro de una transacción, lo que permite que Hibernate sincronice los cambios con la base de datos sin necesidad de cargar previamente la entidad gestionada. Para eliminar un cliente se busca primero por id; si no existe se sale sin hacer nada, y si existe se elimina dentro de una transacción igual que en los métodos anteriores.

Por último, el método `listarComandasDeCliente` sirve para obtener todas las comandas asociadas a un cliente concreto. Aquí se usa una consulta HQL filtrando por el id del cliente, devolviendo así todas las comandas cuyo campo `cliente` apunte al mismo id. En conjunto, esta clase encapsula las operaciones típicas que se realizan sobre clientes y deja la lógica de acceso a datos ordenada y separada del resto de la aplicación.

dao/CamareroDAO.java:

```
package dao;

import modelo.Camarero;
import modelo.Comanda;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.time.LocalDateTime;
import java.util.List;

public class CamareroDAO {

    public void guardar(Camarero c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Camarero obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Camarero.class, id);
        }
    }

    public List<Camarero> listarTodos() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Camarero",
Camarero.class).list();
        }
    }
}
```

```

    public void actualizar(Camarero c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Camarero c = s.find(Camarero.class, id);
            if (c == null) return;

            tx = s.beginTransaction();
            s.remove(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Comanda> listarComandasDeCamarero(Long idCamarero)
    {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            String hql = "FROM Comanda c WHERE c.camarero.id =
:idCam";
            return s.createQuery(hql, Comanda.class)
                .setParameter("idCam", idCamarero)
                .list();
        }
    }
}

```

Este código define la clase CamareroDAO, que se encarga de gestionar todo el acceso a datos relacionado con los camareros dentro de Hibernate. La idea es centralizar en un único sitio las operaciones más comunes sobre la entidad Camarero para que el resto de la aplicación no tenga que preocuparse de cómo funciona exactamente la base de datos ni cómo se abren las sesiones. El método guardar recibe un camarero nuevo y lo persiste dentro de una transacción; si ocurre algún error se revierte todo para que la base de datos no quede en un estado inconsistente.

El método obtener abre una sesión y busca un camarero por su id usando s.find, devolviéndolo si existe. Con listarTodos se realiza una consulta HQL que recupera todos los camareros almacenados. El método actualizar usa merge dentro de una transacción para sincronizar los cambios de un camarero que ya existía previamente, algo bastante común cuando se modifican datos desde un formulario o un menú de consola. Para eliminar funciona de forma parecida: primero se busca el camarero por id, y si se encuentra se borra dentro de una transacción.

Además hay un método extra que permite obtener todas las comandas asociadas a un camarero concreto. Para eso se hace una consulta HQL filtrando por el id del camarero, lo que devuelve la lista de comandas que fueron atendidas por él. Todo el DAO intenta mantener un estilo coherente, abriendo sesiones de manera puntual y manejando las transacciones en los lugares donde realmente son necesarias.

dao/MesaDAO.java:

```
package dao;

import modelo.Mesa;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class MesaDAO {

    public void guardar(Mesa m) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(m);
            tx.commit();
        } catch (Exception e) {
```

```

        if (tx != null) tx.rollback();
        e.printStackTrace();
    }
}

public Mesa obtener(Long id) {
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        return s.find(Mesa.class, id);
    }
}

public List<Mesa> listarTodas() {
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        return s.createQuery("FROM Mesa", Mesa.class).list();
    }
}

public void actualizar(Mesa m) {
    Transaction tx = null;
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        tx = s.beginTransaction();
        s.merge(m);
        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    }
}

public void eliminar(Long id) {
    Transaction tx = null;
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        Mesa m = s.find(Mesa.class, id);
        if (m == null) return;

        tx = s.beginTransaction();
        s.remove(m);
        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
    }
}

```

```

        e.printStackTrace();
    }
}

public void actualizarEstado(Long idMesa, String estado) {
    Transaction tx = null;
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        tx = s.beginTransaction();
        Mesa m = s.find(Mesa.class, idMesa);
        if (m != null) {
            m.setEstado(estado);
            s.merge(m);
        }
        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    }
}

// Listar mesas ocupadas con número de comandas activas
public List<Object[]> mesasOcupadasConCuentaComandas() {
    try (var s =
HibernateUtil.getSessionFactory().openSession()) {
        String hql =
            "SELECT m, COUNT(c) " +
            "FROM Mesa m JOIN m.comandas c " +
            "WHERE m.estado = 'ocupada' " +
            "GROUP BY m";
        return s.createQuery(hql, Object[].class).list();
    }
}
}

```

Este código define la clase MesaDAO, que es la capa de acceso a datos para la entidad Mesa usando Hibernate. La clase permite guardar, obtener, listar, actualizar y eliminar mesas en la base de datos. Para cada operación se abre una sesión de Hibernate y se inicia una transacción. Si la operación se completa correctamente, se hace commit; si ocurre algún error, se hace rollback para deshacer cambios.

El método guardar persiste una nueva mesa en la base de datos. El método obtener recupera una mesa por su id usando s.find. El método listarTodas devuelve todas las mesas mediante una consulta HQL. El método actualizar actualiza los datos de una mesa existente usando merge. El método eliminar busca una mesa por id y, si existe, la elimina de la base de datos.

Además, la clase incluye un método `actualizarEstado` que permite cambiar el estado de una mesa a "libre" o "ocupada". Por último, el método `mesasOcupadasConCuentaComandas` devuelve una lista de mesas que están ocupadas junto con el número de comandas asociadas a cada una, usando una consulta HQL con JOIN y GROUP BY.

`MesaDAO` gestiona todas las operaciones CRUD y algunas consultas específicas sobre las mesas, garantizando que cada operación se ejecute dentro de una transacción para mantener la consistencia de la base de datos, y usando Hibernate para mapear objetos `Mesa` a registros en la tabla correspondiente.

`dao/PlatoDAO.java`:

```
package dao;

import modelo.Plato;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class PlatoDAO {

    public void guardar(Plato p) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(p);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Plato obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Plato.class, id);
        }
    }
}
```

```

    public List<Plato> listarTodos() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Plato", Plato.class).list();
        }
    }

    public void actualizar(Plato p) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(p);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Plato p = s.find(Plato.class, id);
            if (p == null) return;

            tx = s.beginTransaction();
            s.remove(p);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Plato> listarPorCategoria(String categoria) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Plato p WHERE p.categoria =
:cat", Plato.class)
                .setParameter("cat", categoria)
                .list();
        }
    }

```



```

    }
}

```

Este código define la clase PlatoDAO, que se encarga de manejar todas las operaciones de acceso a datos para la entidad Plato usando Hibernate. La clase permite crear, leer, actualizar y eliminar platos en la base de datos. Cada operación abre una sesión de Hibernate y, cuando es necesario, inicia una transacción. Si la operación se completa correctamente, se hace commit, y si hay algún error se hace rollback para deshacer los cambios y evitar inconsistencias.

El método guardar persiste un nuevo plato en la base de datos. El método obtener busca un plato por su id usando s.find. El método listarTodos devuelve todos los platos mediante una consulta HQL. El método actualizar actualiza un plato existente usando merge, y el método eliminar busca un plato por id y, si existe, lo elimina de la base de datos.

Además, la clase tiene un método listarPorCategoria que permite obtener todos los platos de una categoría concreta, como entrada, principal, postre o bebida, usando una consulta HQL con parámetro.

PlatoDAO centraliza toda la lógica de acceso a la base de datos para los platos, asegurando que cada operación se ejecute dentro de una transacción y usando Hibernate para mapear los objetos Plato a la tabla correspondiente.

dao/ComandaDAO.java:

```

package dao;

import modelo.Comanda;
import modelo.ComandaPlato;
import modelo.Plato;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.time.LocalDateTime;
import java.util.List;

public class ComandaDAO {

    public Comanda crearComanda(Comanda comanda) {
        Transaction tx = null;
    }
}

```

```

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();

            // Asegurar que cada item conoce la comanda
            for (ComandaPlato cp : comanda.getItems()) {
                cp.setComanda(comanda);
            }

            s.persist(comanda);
            tx.commit();
            return comanda;

        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }

        return null;
    }

    public Comanda obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Comanda.class, id);
        }
    }

    public List<Comanda> listarTodas() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Comanda",
Comanda.class).list();
        }
    }

    public void actualizar(Comanda comanda) {
        Transaction tx = null;

```

```

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(comanda);
            tx.commit();

        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminarComanda(Long idComanda) {
        Transaction tx = null;

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();

            Comanda c = s.find(Comanda.class, idComanda);
            if (c != null) s.remove(c);

            tx.commit();

        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    // Consultas HQL

    // Listar comandas de una mesa concreta
    public List<Comanda> comandasPorMesa(Long idMesa) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            String hql = ""
SELECT DISTINCT c
FROM Comanda c
LEFT JOIN FETCH c.items cp
LEFT JOIN FETCH cp.plato
WHERE c.mesa.id = :idMesa

```

```

        """;

        return s.createQuery(hql, Comanda.class)
            .setParameter("idMesa", idMesa)
            .list();
    }
}

// Consultar comandas de un cliente mostrando mesa y camarero
public List<Comanda> comandasPorCliente(Long idCliente) {
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
        String hql = "FROM Comanda c WHERE c.cliente.id =
:idCli";
        return s.createQuery(hql, Comanda.class)
            .setParameter("idCli", idCliente)
            .list();
    }
}

// Total facturado por un camarero en un rango de fechas
public Double totalFacturadoPorCamarero(Long idCamarero,
LocalDateTime desde, LocalDateTime hasta) {
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {

        String hql = """
            SELECT SUM(item.cantidad * p.precio)
            FROM Comanda c
            JOIN c.items item
            JOIN item.plato p
            WHERE c.camarero.id = :idCam
            AND c.fechaHora BETWEEN :ini AND :fin
            """;

        Double total = s.createQuery(hql, Double.class)
            .setParameter("idCam", idCamarero)
            .setParameter("ini", desde)
            .setParameter("fin", hasta)
            .uniqueResult();

        return total != null ? total : 0.0;
    }
}

// Total facturado por un cliente en un rango de fechas

```

```

    public Double totalFacturadoPorCliente(Long idCliente,
LocalDateTime desde, LocalDateTime hasta) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {

            String hql = """
                SELECT SUM(item.cantidad * p.precio)
                FROM Comanda c
                JOIN c.items item
                JOIN item.plato p
                WHERE c.cliente.id = :idCli
                AND c.fechaHora BETWEEN :ini AND :fin
            """;

            Double total = s.createQuery(hql, Double.class)
                .setParameter("idCli", idCliente)
                .setParameter("ini", desde)
                .setParameter("fin", hasta)
                .uniqueResult();

            return total != null ? total : 0.0;
        }
    }

    // Total facturado por categoría de plato
    public List<Object[]> totalFacturadoPorCategoria() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {

            String hql = """
                SELECT p.categoria, SUM(cp.cantidad * p.precio)
                FROM Comanda c
                JOIN c.items cp
                JOIN cp.plato p
                GROUP BY p.categoria
            """;

            return s.createQuery(hql, Object[].class).list();
        }
    }
}

```

Este código define la clase ComandaDAO, que se encarga de manejar todas las operaciones de acceso a datos para la entidad Comanda usando Hibernate. La clase permite crear, leer, actualizar y eliminar comandas en la base de datos, así como realizar consultas más específicas como calcular totales facturados o listar comandas según mesa o cliente.

El método crearComanda persiste una nueva comanda en la base de datos y se asegura de que cada item de la comanda conozca a qué comanda pertenece, manejando todo dentro de una transacción para mantener la integridad de los datos. El método obtener permite recuperar una comanda por su id, y listarTodas devuelve todas las comandas existentes. actualizar se usa para modificar una comanda existente, y eliminarComanda elimina una comanda por su id si existe.

También incluye consultas HQL más avanzadas. El método comandasPorMesa devuelve todas las comandas de una mesa concreta y carga los platos de cada item para evitar problemas de lazy loading. comandasPorCliente lista todas las comandas de un cliente. Los métodos totalFacturadoPorCamarero y totalFacturadoPorCliente calculan el total facturado por un camarero o un cliente dentro de un rango de fechas, multiplicando la cantidad de cada plato por su precio y sumando los resultados. Por último, totalFacturadoPorCategoria calcula el total facturado agrupado por la categoría de cada plato.

ComandaDAO centraliza toda la logica de acceso a datos y consultas de comandas, asegurando que cada operación se ejecute dentro de una transacción y usando Hibernate para mapear los objetos a la base de datos.

util/HibernateUtil.java:

```
package util;

import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;

public class HibernateUtil {
    private static final SessionFactory sessionFactory =
        buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            StandardServiceRegistry registry = new
StandardServiceRegistryBuilder()
                .configure() // busca hibernate.cfg.xml en
resources
                .build();
            Metadata metadata = new
MetadataSources(registry).getMetadataBuilder().build();
            return metadata.getSessionFactoryBuilder().build();
        } catch (Exception e) {
            System.err.println("Initial SessionFactory creation
failed: " + e);
            throw new ExceptionInInitializerError(e);
        }
    }

    public static SessionFactory getSessionFactory() { return
sessionFactory; }
    public static void shutdown() { getSessionFactory().close(); }
}
```

Este código define la clase `HibernateUtil`, que sirve como utilidad para manejar la sesión de Hibernate en toda la aplicación. La clase crea una única instancia de `SessionFactory`, que es el objeto central que Hibernate usa para abrir sesiones con la base de datos.

El método `buildSessionFactory` se encarga de construir esta `SessionFactory`. Primero crea un `StandardServiceRegistry` leyendo la configuración de Hibernate desde el archivo `hibernate.cfg.xml` que debe estar en la carpeta de `resources`. Luego construye un `Metadata` a partir de ese registry, que contiene toda la información de mapeos de entidades y configuración, y finalmente construye la `SessionFactory`. Si ocurre algún error durante este proceso, se imprime un mensaje de error y se lanza una excepción de inicialización.

La clase también provee el método `getSessionFactory` para obtener la `SessionFactory` ya construida, y `shutdown` para cerrar la `SessionFactory` al final del programa y liberar los recursos de conexión a la base de datos. Concluyendo, `HibernateUtil` centraliza la configuración de Hibernate y proporciona una forma fácil de acceder a la sesión en cualquier parte de la aplicación.

hibernate.cfg.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.connection.driver_class">org.sqlite.JDBC</property
>
        <property
name="hibernate.connection.url">jdbc:sqlite:restaurante.db</proper
ty>
        <property
name="hibernate.dialect">org.hibernate.community.dialect.SQLiteDia
lect</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="modelo.Mesa"/>
        <mapping class="modelo.Plato"/>
        <mapping class="modelo.Camarero"/>
        <mapping class="modelo.Cliente"/>
        <mapping class="modelo.Comanda"/>
        <mapping class="modelo.ComandaPlato"/>
    </session-factory>
</hibernate-configuration>
```

Este archivo es la configuración de Hibernate en XML para la aplicación del restaurante. Define cómo Hibernate se va a conectar a la base de datos y cómo manejará las entidades.

Primero, se especifica el driver JDBC que se va a usar, en este caso `org.sqlite.JDBC`, y la URL de la base de datos, que apunta a un fichero SQLite llamado `restaurante.db`. Luego se indica el dialecto de Hibernate, que es `SQLiteDialect`, para que Hibernate sepa cómo generar las consultas SQL correctas para SQLite.

Se activan también dos opciones útiles para depuración: `show_sql` en `true` hace que se muestren en consola las sentencias SQL que Hibernate ejecuta, y `format_sql` las muestra formateadas para que sean más legibles.

La propiedad `hbm2ddl.auto` con valor `update` indica que Hibernate actualizará automáticamente la estructura de la base de datos según los cambios en las entidades, sin borrar los datos existentes.

Finalmente, se listan todas las clases de modelo que Hibernate debe mapear a tablas de la base de datos: `Mesa`, `Plato`, `Camarero`, `Cliente`, `Comanda` y `ComandaPlato`. Cada clase corresponde a una tabla y Hibernate se encarga de gestionar las relaciones y operaciones sobre ellas.

pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>act3ut3</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>26</maven.compiler.source>
    <maven.compiler.target>26</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <hibernate.version>7.1.0.Final</hibernate.version>
    <sqlite.jdbc.version>3.50.3.0</sqlite.jdbc.version>
    <slf4j.version>2.0.17</slf4j.version>
  </properties>

  <dependencies>
    <!-- Hibernate core (incluye implementación JPA) -->
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
```

```

        <version>${hibernate.version}</version>
    </dependency>

    <!-- Community Dialects (SQLite) -->
    <dependency>
        <groupId>org.hibernate.orm</groupId>
        <artifactId>hibernate-community-dialects</artifactId>
        <version>${hibernate.version}</version>
    </dependency>

    <!-- SQLite JDBC -->
    <dependency>
        <groupId>org.xerial</groupId>
        <artifactId>sqlite-jdbc</artifactId>
        <version>${sqlite.jdbc.version}</version>
    </dependency>

    <!-- SLF4J Simple -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>${slf4j.version}</version>
    </dependency>
</dependencies>
</project>

```

El archivo pom.xml es la configuración de Maven para el proyecto del restaurante y define cómo se va a compilar y qué dependencias se van a usar.

Primero se indica el groupId, artifactId y version del proyecto, que son los identificadores únicos dentro de Maven. Después, en la sección de properties, se especifica que el código se compila para Java 26 y se usa UTF-8 como codificación de caracteres. También se definen las versiones de Hibernate, del driver JDBC de SQLite y de SLF4J que se van a usar.

En la sección de dependencies se incluyen todas las librerías necesarias. Se añade hibernate-core que incluye la implementación de JPA y las funciones principales de Hibernate. También se incluye hibernate-community-dialects, que sirve para poder usar SQLite con Hibernate, ya que no viene soportado por defecto. Luego está la dependencia de sqlite-jdbc, que permite a Java conectarse a bases de datos SQLite, y finalmente slf4j-simple, que es una implementación ligera de SLF4J para que Hibernate pueda mostrar logs de ejecución de manera sencilla.

Este archivo asegura que Maven descargue todas las librerías necesarias y que el proyecto se compile correctamente con Java 26, Hibernate 7.1 y SQLite, permitiendo que la aplicación funcione sin problemas de dependencias.

Capturas de pantalla

```
--- MENU RESTAURANTE ---  
1. Insertar datos de ejemplo  
2. Crear comanda (con items)  
3. Listar comandas de una mesa  
4. Listar platos por categoria  
5. Total facturado por camarero (rango)  
6. Total facturado por cliente (rango)  
7. Comandas de cliente (mostrar mesa y camarero)  
8. Listar mesas ocupadas con n° comandas  
9. Actualizar estado mesa  
10. Eliminar comanda  
11. Total facturado agrupado por categoría  
0. Salir  
Opción: 1
```

```
Hibernate:
    create table camareros (
        id integer,
        dni varchar(255),
        nombre varchar(255),
        primary key (id)
    )
Hibernate:
    create table clientes (
        id integer,
        email varchar(255),
        nombre varchar(255),
        telefono varchar(255),
        primary key (id)
    )
Hibernate:
    create table comanda_plato (
        cantidad integer not null,
        comanda_id bigint not null,
        plato_id bigint not null,
        primary key (comanda_id, plato_id)
    )
Hibernate:
    create table comandas (
        id integer,
        fechaHora timestamp,
        camarero_id bigint,
        cliente_id bigint,
        mesa_id bigint,
        primary key (id)
    )
Hibernate:
    create table mesas (
        id integer,
        capacidad integer not null,
        estado varchar(255),
        numero integer not null unique,
        primary key (id)
    )
Hibernate:
    create table platos (
        id integer
```

```
Hibernate:
    create table platos (
        id integer,
        categoria varchar(255),
        descripcion varchar(255),
        nombre varchar(255),
        precio float not null,
        primary key (id)
    )
Hibernate:
    alter table mesas
        drop constraint UKj0e0hutvqcecq56ogql18feqw
Hibernate:
    insert
    into
        mesas
        (capacidad, estado, numero)
    values
        (?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        mesas
        (capacidad, estado, numero)
    values
        (?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        platos
        (categoria, descripcion, nombre, precio)
    values
        (?, ?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
```

```
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        platos
        (categoria, descripcion, nombre, precio)
    values
        (?, ?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        platos
        (categoria, descripcion, nombre, precio)
    values
        (?, ?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        platos
        (categoria, descripcion, nombre, precio)
    values
        (?, ?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        camareros
        (dni, nombre)
    values
        (?, ?)
Hibernate:
    select
```

```
        platos
        (categoria, descripcion, nombre, precio)
    values
        (?, ?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        camareros
        (dni, nombre)
    values
        (?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        clientes
        (email, nombre, telefono)
    values
        (?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Datos de ejemplo insertados.
```

```
--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoría
0. Salir
Opción: |
```

Main.java restaurante.db

Tables Database Metadata

Table: camareros Page: 0

id	dni	nombre
1	1234567...	Laura

Main.java restaurante.db

Tables Database Metadata

Table: clientes Page: 0

id	email	nombre	telefono
1	c@ejemp...	Carlos	600111222

Main.java restaurante.db

Tables Database Metadata

Table: mesas Page: 0

id	capacid...	estado	numero
1	4	libre	1
2	2	ocupada	2

Main.java restaurante.db ×

Tables Database Metadata

Table: platos Page: 0 Ju

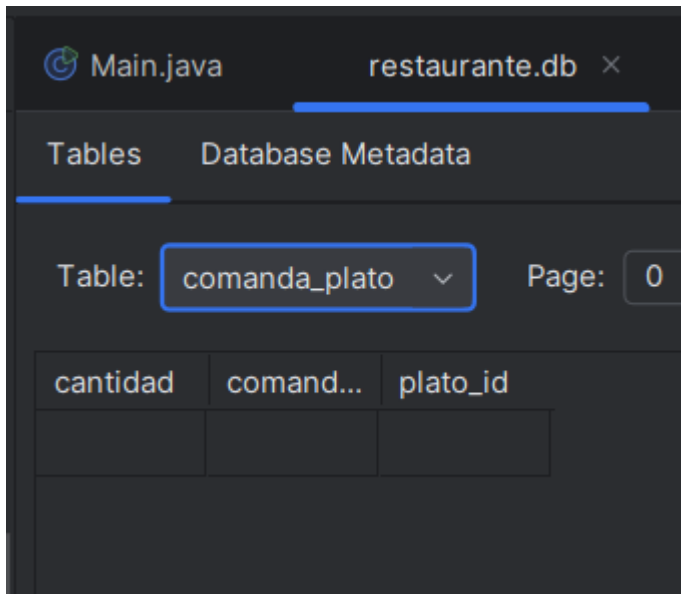
id	categoria	descrip...	nombre	precio
1	entrada	Ensalada ...	Ensalada	6.5
2	principal	Solomillo...	Solomillo	15.0
3	postre	Tiramisu ...	Tiramisu	5.0
4	bebida	Cerveza ...	Cerveza	2.5

Main.java restaurante.db ×

Tables Database Metadata

Table: comandas Page: 0 Ju

id	fechaH...	camare...	cliente_id	mesa_id



```
--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 2
ID mesa: 1
ID camarero: 1
ID cliente: 1
```

```
Hibernate:
  select
    m1_0.id,
    m1_0.capacidad,
    m1_0.estado,
    m1_0.numero
  from
    mesas m1_0
  where
    m1_0.id=?
Hibernate:
  select
    c1_0.id,
    c1_0.dni,
    c1_0.nombre
  from
    camareros c1_0
  where
    c1_0.id=?
Hibernate:
  select
    c1_0.id,
    c1_0.email,
    c1_0.nombre,
    c1_0.telefono
  from
    clientes c1_0
  where
    c1_0.id=?
Hibernate:
  select
    m1_0.id,
    m1_0.capacidad,
    c1_0.mesa_id,
    c1_0.id,
    c2_0.id,
    c2_0.dni,
    c2_0.nombre
```

```
Hibernate:
  select
    m1_0.id,
    m1_0.capacidad,
    c1_0.mesa_id,
    c1_0.id,
    c2_0.id,
    c2_0.dni,
    c2_0.nombre,
    c3_0.id,
    c3_0.email,
    c3_0.nombre,
    c3_0.telefono,
    c1_0.fechaHora,
    m1_0.estado,
    m1_0.numero
  from
    mesas m1_0
  left join
    comandas c1_0
      on m1_0.id=c1_0.mesa_id
  left join
    camareros c2_0
      on c2_0.id=c1_0.camarero_id
  left join
    clientes c3_0
      on c3_0.id=c1_0.cliente_id
  where
    m1_0.id=?
Hibernate:
  update
    mesas
  set
    capacidad=?,
    estado=?,
    numero=?
  where
    id=?
¿Cuántos platos en la comanda? 4
```

```
¿Cuántos platos en la comanda? 4
ID plato: 1
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
ID plato: 2
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
ID plato: 3
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
```

```
ID plato: 3
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
ID plato: 4
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
Hibernate:
    insert
    into
        comandas
        (camarero_id, cliente_id, fechaHora, mesa_id)
    values
        (?, ?, ?, ?)
Hibernate:
    select
        last_insert_rowid()
Hibernate:
    insert
    into
        comanda_plato
        (cantidad, comanda_id, plato_id)
    values
        (?, ?, ?)
```

```
        into
            comanda_plato
            (cantidad, comanda_id, plato_id)
        values
            (?, ?, ?)
Hibernate:
        insert
        into
            comanda_plato
            (cantidad, comanda_id, plato_id)
        values
            (?, ?, ?)
Hibernate:
        insert
        into
            comanda_plato
            (cantidad, comanda_id, plato_id)
        values
            (?, ?, ?)
Hibernate:
        insert
        into
            comanda_plato
            (cantidad, comanda_id, plato_id)
        values
            (?, ?, ?)
Comanda creada con éxito:
Comanda{id=1, fechaHora=2025-12-05T21:40:33.981296700, mesa=1, camarero=Laura, cliente=Carlos}

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción:
```

Table: mesas Page: 0

id	capacid...	estado	numero
1	4	ocupada	1
2	2	ocupada	2

Table: comandas Page: 0

id	fechaH...	camare...	cliente_id	mesa_id
1	1764970...	1	1	1

Table: comanda_plato Page:

cantidad	comand...	plato_id
1	1	3
1	1	4
1	1	2
1	1	1


```
--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 3
ID mesa: 1
Hibernate:
    select
        distinct c1_0.id,
        c1_0.camarero_id,
        c1_0.cliente_id,
        c1_0.fechaHora,
        i1_0.comanda_id,
        i1_0.plato_id,
        i1_0.cantidad,
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio,
        c1_0.mesa_id
    from
        comandas c1_0
    left join
        comanda_plato i1_0
        on c1_0.id=i1_0.comanda_id
    left join
        platos p1_0
        on p1_0.id=i1_0.plato_id
    where
        c1_0.mesa_id=?
Hibernate:
    select
        c1_0.id,
        c1_0.dni,
```

```
Hibernate:
    select
        c1_0.id,
        c1_0.dni,
        c1_0.nombre
    from
        camareros c1_0
    where
        c1_0.id=?
Hibernate:
    select
        c1_0.id,
        c1_0.email,
        c1_0.nombre,
        c1_0.telefono
    from
        clientes c1_0
    where
        c1_0.id=?
Hibernate:
    select
        m1_0.id,
        m1_0.capacidad,
        m1_0.estado,
        m1_0.numero
    from
        mesas m1_0
    where
        m1_0.id=?
Comanda{id=1, fechaHora=2025-12-05T21:40:33.981, mesa=1, camarero=Laura, cliente=Carlos}
    - Ensalada x1
    - Cerveza x1
    - Solomillo x1
    - Tiramisu x1

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
```

```

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 4
Categoria (entrada/principal/postre/bebida): postre
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.categoria=?
Plato{id=3, nombre='Tiramisu', precio=5.0, categoria=postre}

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción:

```

```
Opción: 5
ID camarero: 1
Desde (yyyy-MM-ddTHH:mm) (ejemplo 2025-12-01T14:30): 2025-12-05T21:30
Hasta (yyyy-MM-ddTHH:mm): 2025-12-05T21:45
Hibernate:
    select
        sum(i1_0.cantidad*p1_0.precio)
    from
        comandas c1_0
    join
        comanda_plato i1_0
        on c1_0.id=i1_0.comanda_id
    join
        platos p1_0
        on p1_0.id=i1_0.plato_id
    where
        c1_0.camarero_id=?
        and c1_0.fechaHora between ? and ?
Facturado: 29.0 €

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción:
.....
```

```
Opción: 6
ID cliente: 1
Desde (yyyy-MM-ddTHH:mm) (ejemplo 2025-12-01T14:30): 2025-12-05T21:30
Hasta (yyyy-MM-ddTHH:mm): 2025-12-05T21:45
Hibernate:
    select
        sum((i1_0.cantidad*p1_0.precio))
    from
        comandas c1_0
    join
        comanda_plato i1_0
        on c1_0.id=i1_0.comanda_id
    join
        platos p1_0
        on p1_0.id=i1_0.plato_id
    where
        c1_0.cliente_id=?
        and c1_0.fechaHora between ? and ?
Total facturado por cliente: 29.0 €

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoría
0. Salir
Opción: |
```

```
Opción: 7
ID cliente: 1
Hibernate:
    select
        c1_0.id,
        c1_0.camarero_id,
        c1_0.cliente_id,
        c1_0.fechaHora,
        c1_0.mesa_id
    from
        comandas c1_0
    where
        c1_0.cliente_id=?
Hibernate:
    select
        c1_0.id,
        c1_0.dni,
        c1_0.nombre
    from
        camareros c1_0
    where
        c1_0.id=?
Hibernate:
    select
        c1_0.id,
        c1_0.email,
        c1_0.nombre,
        c1_0.telefono
    from
        clientes c1_0
    where
        c1_0.id=?
Hibernate:
    select
        m1_0.id,
        m1_0.capacidad,
        m1_0.estado,
        m1_0.numero
    from
        mesas m1_0
    where
        m1_0.id=?
Comanda{id=1, fechaHora=2025-12-05T21:40:33.981, mesa=1, camarero=Laura, cliente=Carlos} | Mesa: 1 | Camarero: Laura

--- MENU RESTAURANTE ---
```

```
Opción: 8
Hibernate:
    select
        m1_0.id,
        m1_0.capacidad,
        m1_0.estado,
        m1_0.numero,
        count(c1_0.id)
    from
        mesas m1_0
    join
        comandas c1_0
        on m1_0.id=c1_0.mesa_id
    where
        m1_0.estado='ocupada'
    group by
        m1_0.id,
        m1_0.capacidad,
        m1_0.estado,
        m1_0.numero
Mesa{id=1, numero=1, capacidad=4, estado=ocupada} -> n° comandas: 1

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoría
0. Salir
Opción: |
```

Main.java restaurante.db ×

Tables Database Metadata

Table: Page:

id	capacid...	estado	numero
1	4	ocupada	1
2	2	ocupada	2


```
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 9
ID mesa: 2
Nuevo estado (libre/ocupada): libre
Hibernate:
    select
        m1_0.id,
        m1_0.capacidad,
        m1_0.estado,
        m1_0.numero
    from
        mesas m1_0
    where
        m1_0.id=?
Hibernate:
    update
        mesas
    set
        capacidad=?,
        estado=?,
        numero=?
    where
        id=?
Estado actualizado.

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción:
```

Main.java restaurante.db

Tables Database Metadata

Table: **mesas** Page: 0

id	capacid...	estado	numero
1	4	ocupada	1
2	2	libre	2

Table: **comandas** Page: 0

id	fechaH...	camare...	cliente_id	mesa_id
1	1764970...	1	1	1

Table: **comanda_plato** Page: 0

cantidad	comand...	plato_id
1	1	3
1	1	4
1	1	2
1	1	1

```
Opción: 10
ID comanda: 1
Hibernate:
    select
        c1_0.id,
        c2_0.id,
        c2_0.dni,
        c2_0.nombre,
        c3_0.id,
        c3_0.email,
        c3_0.nombre,
        c3_0.telefono,
        c1_0.fechaHora,
        m1_0.id,
        m1_0.capacidad,
        m1_0.estado,
        m1_0.numero
    from
        comandas c1_0
    left join
        camareros c2_0
        on c2_0.id=c1_0.camarero_id
    left join
        clientes c3_0
        on c3_0.id=c1_0.cliente_id
    left join
        mesas m1_0
        on m1_0.id=c1_0.mesa_id
    where
        c1_0.id=?
Hibernate:
    select
        i1_0.comanda_id,
        i1_0.plato_id,
        i1_0.cantidad,
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        comanda_plato i1_0
    left join
```

```
    left join
      platos p1_0
        on p1_0.id=i1_0.plato_id
    where
      i1_0.comanda_id=?
Hibernate:
    delete
    from
      comanda_plato
    where
      comanda_id=?
      and plato_id=?
Hibernate:
    delete
    from
      comanda_plato
    where
      comanda_id=?
      and plato_id=?
Hibernate:
    delete
    from
      comanda_plato
    where
      comanda_id=?
      and plato_id=?
Hibernate:
    delete
    from
      comandas
    where
      id=?
Comanda eliminada (si existia).
```

Table: **comandas** Page: 0

id	fechaH...	camare...	cliente_id	mesa_id

Table: **comanda_plato** Page

cantidad	comand...	plato_id

```
10-?
Comanda eliminada (si existia).

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 11
Hibernate:
    select
        p1_0.categoria,
        sum((i1_0.cantidad*p1_0.precio))
    from
        comandas c1_0
    join
        comanda_plato i1_0
        on c1_0.id=i1_0.comanda_id
    join
        platos p1_0
        on p1_0.id=i1_0.plato_id
    group by
        p1_0.categoria

--- Total facturado por categoria ---

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 4
```

```
--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoría
0. Salir
Opción: 2
ID mesa: 1
ID camarero: 1
ID cliente: 1
```

```
-
¿Cuántos platos en la comanda? 4
ID plato: 1
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
ID plato: 2
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
ID plato: 3
Cantidad: 1
Hibernate:
    select
        p1_0.id,
        p1_0.categoria,
        p1_0.descripcion,
        p1_0.nombre,
        p1_0.precio
    from
        platos p1_0
    where
        p1_0.id=?
ID plato: 4
Cantidad: 1
```



```

Comanda creada con éxito:
Comanda{id=1, fechaHora=2025-12-05T22:15:30.865497900, mesa=1, camarero=Laura, cliente=Carlos}

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoria
0. Salir
Opción: 11
Hibernate:
    select
        p1_0.categoria,
        sum((i1_0.cantidad*p1_0.precio))
    from
        comandas c1_0
    join
        comanda_plato i1_0
        on c1_0.id=i1_0.comanda_id
    join
        platos p1_0
        on p1_0.id=i1_0.plato_id
    group by
        p1_0.categoria

--- Total facturado por categoria ---
bebida: 2.5 €
entrada: 6.5 €
postre: 5.0 €
principal: 15.0 €

--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa

```

```
--- MENU RESTAURANTE ---
1. Insertar datos de ejemplo
2. Crear comanda (con items)
3. Listar comandas de una mesa
4. Listar platos por categoria
5. Total facturado por camarero (rango)
6. Total facturado por cliente (rango)
7. Comandas de cliente (mostrar mesa y camarero)
8. Listar mesas ocupadas con n° comandas
9. Actualizar estado mesa
10. Eliminar comanda
11. Total facturado agrupado por categoría
0. Salir
Opción: 0

Process finished with exit code 0
|
```

Código completo

Main.java:

```
import modelo.*;
import dao.*;
import util.HibernateUtil;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        MesaDAO mesaDAO = new MesaDAO();
        PlatoDAO platoDAO = new PlatoDAO();
        CamareroDAO camareroDAO = new CamareroDAO();
        ClienteDAO clienteDAO = new ClienteDAO();
        ComandaDAO comandaDAO = new ComandaDAO();

        Scanner sc = new Scanner(System.in);
        int opt;

        do {
            System.out.println("\n--- MENU RESTAURANTE ---");
            System.out.println("1. Insertar datos de ejemplo");
            System.out.println("2. Crear comanda (con items)");
            System.out.println("3. Listar comandas de una mesa");
            System.out.println("4. Listar platos por categoria");
            System.out.println("5. Total facturado por camarero
(rango)");
            System.out.println("6. Total facturado por cliente
(rango)");
            System.out.println("7. Comandas de cliente (mostrar
mesa y camarero)");
            System.out.println("8. Listar mesas ocupadas con n°
comandas");
            System.out.println("9. Actualizar estado mesa");
            System.out.println("10. Eliminar comanda");
            System.out.println("11. Total facturado agrupado por
categoria");
            System.out.println("0. Salir");
            System.out.print("Opción: ");
            opt = Integer.parseInt(sc.nextLine());

            switch (opt) {
```

```

        case 1 -> {
            Mesa m1 = new Mesa(1, 4, "libre");
            Mesa m2 = new Mesa(2, 2, "ocupada");
            mesaDAO.guardar(m1);
            mesaDAO.guardar(m2);

            platoDAO.guardar(new Plato("Ensalada",
"Ensalada mixta", 6.5, "entrada"));
            platoDAO.guardar(new Plato("Solomillo",
"Solomillo a la brasa", 15.0, "principal"));
            platoDAO.guardar(new Plato("Tiramisu",
"Tiramisu de piña", 5.0, "postre"));
            platoDAO.guardar(new Plato("Cerveza", "Cerveza
Dorada", 2.5, "bebida"));

            camareroDAO.guardar(new Camarero("Laura",
"12345678Z"));
            clienteDAO.guardar(new Cliente("Carlos",
"600111222", "c@ejemplo.com"));

            System.out.println("Datos de ejemplo
insertados.");
        }

        case 2 -> {
            System.out.print("ID mesa: ");
            Long idMesa = Long.parseLong(sc.nextLine());
            System.out.print("ID camarero: ");
            Long idCam = Long.parseLong(sc.nextLine());
            System.out.print("ID cliente: ");
            Long idCli = Long.parseLong(sc.nextLine());

            Mesa mesa = mesaDAO.obtener(idMesa);
            Camarero camarero = camareroDAO.obtener(idCam);
            Cliente cliente = clienteDAO.obtener(idCli);

            if (mesa == null || camarero == null || cliente
== null) {
                System.out.println("IDs inválidos. No se
puede crear la comanda.");
                break;
            }

            // La mesa pasa a "ocupada" al crear una
comanda

            mesa.setEstado("ocupada");
            mesaDAO.actualizar(mesa);

```

```

        Comanda comanda = new
Comanda(LocalDate.now(), mesa, camarero, cliente);

        System.out.print("¿Cuántos platos en la
comanda? ");

        int n = Integer.parseInt(sc.nextLine());

        for (int i = 0; i < n; i++) {
            System.out.print("ID plato: ");
            Long idPlato =
Long.parseLong(sc.nextLine());
            System.out.print("Cantidad: ");
            int cantidad =
Integer.parseInt(sc.nextLine());

            Plato plato = platoDAO.obtener(idPlato);

            if (plato != null) {
                ComandaPlato cp = new ComandaPlato();
                cp.setPlato(plato);
                cp.setCantidad(cantidad);

                comanda.addItem(cp);
            } else {
                System.out.println("Plato no
encontrado: " + idPlato);
            }
        }

        comandaDAO.crearComanda(comanda);

        System.out.println("Comanda creada con
éxito:");

        System.out.println(comanda);
    }

    case 3 -> {
        System.out.print("ID mesa: ");
        Long id = Long.parseLong(sc.nextLine());
        List<Comanda> lista =
comandaDAO.comandasPorMesa(id);

        lista.forEach(c -> {
            System.out.println(c);
            c.getItems().forEach(it ->
                System.out.println("    - " +
it.getPlato().getNombre() + " x" + it.getCantidad())
            );
        });
    }
}

```

```

        });
    }

    case 4 -> {
        System.out.print("Categoria
(entrada/principal/postre/bebida): ");
        String cat = sc.nextLine();

        platoDAO.listarPorCategoria(cat).forEach(System.out::println);
    }

    case 5 -> {
        System.out.print("ID camarero: ");
        Long idC = Long.parseLong(sc.nextLine());
        System.out.print("Desde (yyyy-MM-ddTHH:mm)
(ejemplo 2025-12-01T14:30): ");
        LocalDateTime desde =
LocalDateTime.parse(sc.nextLine());
        System.out.print("Hasta (yyyy-MM-ddTHH:mm): ");
        LocalDateTime hasta =
LocalDateTime.parse(sc.nextLine());

        Double total =
comandaDAO.totalFacturadoPorCamarero(idC, desde, hasta);
        System.out.println("Facturado: " + (total ==
null ? 0 : total) + " €");
    }

    case 6 -> {
        System.out.print("ID cliente: ");
        Long idCl = Long.parseLong(sc.nextLine());
        System.out.print("Desde (yyyy-MM-ddTHH:mm)
(ejemplo 2025-12-01T14:30): ");
        LocalDateTime desde =
LocalDateTime.parse(sc.nextLine());
        System.out.print("Hasta (yyyy-MM-ddTHH:mm): ");
        LocalDateTime hasta =
LocalDateTime.parse(sc.nextLine());

        Double total =
comandaDAO.totalFacturadoPorCliente(idCl, desde, hasta);

        System.out.println("Total facturado por
cliente: " + (total == null ? 0 : total) + " €");
    }

    case 7 -> {
        System.out.print("ID cliente: ");

```

```

        Long idCl = Long.parseLong(sc.nextLine());
        List<Comanda> cs =
comandaDAO.comandasPorCliente(idCl);

        cs.forEach(c ->
            System.out.println(
                c +
                    " | Mesa: " +
(c.getMesa() != null ? c.getMesa().getNumero() : "-") +
                    " | Camarero: " +
(c.getCamarero() != null ? c.getCamarero().getNombre() : "-")
                )
            );
        }

        case 8 -> {
            List<Object[]> res =
mesaDAO.mesasOcupadasConCuentaComandas();
            res.forEach(row -> {
                Mesa m = (Mesa) row[0];
                Long contador = (Long) row[1];
                System.out.println(m + " -> nº comandas: "
+ contador);
            });
        }

        case 9 -> {
            System.out.print("ID mesa: ");
            Long idM = Long.parseLong(sc.nextLine());
            System.out.print("Nuevo estado (libre/ocupada):
");

            String est = sc.nextLine();
            mesaDAO.actualizarEstado(idM, est);
            System.out.println("Estado actualizado.");
        }

        case 10 -> {
            System.out.print("ID comanda: ");
            Long idCo = Long.parseLong(sc.nextLine());
            comandaDAO.eliminarComanda(idCo);
            System.out.println("Comanda eliminada (si
existía).");
        }

        case 11 -> {
            List<Object[]> lista =
comandaDAO.totalFacturadoPorCategoria();

```

```

        System.out.println("\n--- Total facturado por
categoría ---");
        for (Object[] row : lista) {
            String categoria = (String) row[0];
            Double total = (Double) row[1];
            System.out.println(categoria + ": " +
(total == null ? 0 : total) + " €");
        }
    }

    } while (opt != 0);

    sc.close();
    HibernateUtil.shutdown();
}
}

```

modelo/Camarero.java:

```

package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "camareros")
public class Camarero {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String dni;

    @OneToMany(mappedBy = "camarero", cascade = CascadeType.ALL)
    private Set<Comanda> comandas = new HashSet<>();

    public Camarero() {}
    public Camarero(String nombre, String dni) { this.nombre =
nombre; this.dni = dni; }

    // getters y setters
    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}

```



```

public String getDni() { return dni; }
public void setDni(String dni) { this.dni = dni; }
public Set<Comanda> getComandas() { return comandas; }

@Override
public String toString() { return "Camarero{id=" + id + ",
nombre='" + nombre + "', dni='" + dni + "'}"; }
}

```

modelo/Cliente.java:

```

package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "clientes")
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String telefono;
    private String email;

    @OneToMany(mappedBy = "cliente", cascade = CascadeType.ALL)
    private Set<Comanda> comandas = new HashSet<>();

    public Cliente() {}
    public Cliente(String nombre, String telefono, String email) {
        this.nombre = nombre; this.telefono = telefono; this.email
= email;
    }

    // getters y setters
    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTelefono() { return telefono; }
    public void setTelefono(String telefono) { this.telefono =
telefono; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public Set<Comanda> getComandas() { return comandas; }
}

```

```

    @Override
    public String toString() { return "Cliente{id=" + id + ",
nombre='" + nombre + "'}"; }
}

```

modelo/Mesa.java:

```

package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "mesas")
public class Mesa {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private int numero;

    private int capacidad;

    // "libre" / "ocupada"
    private String estado;

    @OneToMany(mappedBy = "mesa", cascade = CascadeType.ALL,
orphanRemoval = true)
    private Set<Comanda> comandas = new HashSet<>();

    public Mesa() {}
    public Mesa(int numero, int capacidad, String estado) {
        this.numero = numero;
        this.capacidad = capacidad;
        this.estado = estado;
    }

    // getters y setters
    public Long getId() { return id; }
    public int getNumero() { return numero; }
    public void setNumero(int numero) { this.numero = numero; }
    public int getCapacidad() { return capacidad; }
    public void setCapacidad(int capacidad) { this.capacidad =
capacidad; }
}

```

```

public String getEstado() { return estado; }
public void setEstado(String estado) { this.estado = estado; }
public Set<Comanda> getComandas() { return comandas; }

@Override
public String toString() {
    return "Mesa{id=" + id + ", numero=" + numero + ",
capacidad=" + capacidad + ", estado=" + estado + "}";
}
}

```

modelo/Plato.java:

```

package modelo;

import jakarta.persistence.*;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "platos")
public class Plato {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String descripcion;
    private double precio;

    // entrada, principal, postre, bebida
    private String categoria;

    // relación inversa con ComandaPlato
    @OneToMany(mappedBy = "plato", cascade = CascadeType.ALL,
orphanRemoval = true)
    private Set<ComandaPlato> items = new HashSet<>();

    public Plato() {}
    public Plato(String nombre, String descripcion, double precio,
String categoria) {
        this.nombre = nombre; this.descripcion = descripcion;
this.precio = precio; this.categoria = categoria;
    }

    // getters y setters
    public Long getId() { return id; }

```

```

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) {
this.descripcion = descripcion; }
    public double getPrecio() { return precio; }
    public void setPrecio(double precio) { this.precio = precio; }
    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria =
categoria; }
    public Set<ComandaPlato> getItems() { return items; }

    @Override
    public String toString() {
        return "Plato{id=" + id + ", nombre='" + nombre + "',
precio=" + precio + ", categoria=" + categoria + "}";
    }
}

```

modelo/Comanda.java:

```

package modelo;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "comandas")
public class Comanda {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // fecha y hora de la comanda
    private LocalDateTime fechaHora;

    @ManyToOne
    @JoinColumn(name = "mesa_id")
    private Mesa mesa;

    @ManyToOne
    @JoinColumn(name = "camarero_id")
    private Camarero camarero;

    @ManyToOne

```

```

@JoinColumn(name = "cliente_id")
private Cliente cliente;

// items (plato + cantidad) - entidad intermedia
@OneToMany(mappedBy = "comanda", cascade = CascadeType.ALL,
orphanRemoval = true)
private Set<ComandaPlato> items = new HashSet<>();

public Comanda() {}
public Comanda(LocalDateTime fechaHora, Mesa mesa, Camarero
camarero, Cliente cliente) {
    this.fechaHora = fechaHora; this.mesa = mesa; this.camarero
= camarero; this.cliente = cliente;
}

// helpers para gestionar la relación bi-direccional
public void addItem(ComandaPlato item) {
    items.add(item);
    item.setComanda(this);
}
public void removeItem(ComandaPlato item) {
    items.remove(item);
    item.setComanda(null);
}

// getters y setters
public Long getId() { return id; }
public LocalDateTime getFechaHora() { return fechaHora; }
public void setFechaHora(LocalDateTime fechaHora) {
this.fechaHora = fechaHora; }
public Mesa getMesa() { return mesa; }
public void setMesa(Mesa mesa) { this.mesa = mesa; }
public Camarero getCamarero() { return camarero; }
public void setCamarero(Camarero camarero) { this.camarero =
camarero; }
public Cliente getCliente() { return cliente; }
public void setCliente(Cliente cliente) { this.cliente =
cliente; }
public Set<ComandaPlato> getItems() { return items; }

@Override
public String toString() {
    return "Comanda{id=" + id + ", fechaHora=" + fechaHora + ",
mesa=" + (mesa != null ? mesa.getNumero() : null)
        + ", camarero=" + (camarero != null ?
camarero.getNombre() : null) + ", cliente=" + (cliente != null ?
cliente.getNombre() : null) + "}";
}

```

}

modelo/ComandaPlato.java:

```

package modelo;

import jakarta.persistence.*;

@Entity
@Table(name = "comanda_plato")
public class ComandaPlato {

    @EmbeddedId
    private ComandaPlatoId id = new ComandaPlatoId();

    @ManyToOne
    @MapsId("comandaId")
    @JoinColumn(name = "comanda_id")
    private Comanda comanda;

    @ManyToOne
    @MapsId("platoId")
    @JoinColumn(name = "plato_id")
    private Plato plato;

    private int cantidad;

    public ComandaPlato() {}
    public ComandaPlato(Comanda comanda, Plato plato, int cantidad)
    {
        this.comanda = comanda; this.plato = plato; this.cantidad =
cantidad;
        if (comanda != null && plato != null) {
            this.id = new ComandaPlatoId(comanda.getId(),
plato.getId());
        }
    }

    // getters y setters
    public ComandaPlatoId getId() { return id; }
    public Comanda getComanda() { return comanda; }
    public void setComanda(Comanda comanda) { this.comanda =
comanda; }
    public Plato getPlato() { return plato; }
    public void setPlato(Plato plato) { this.plato = plato; }
    public int getCantidad() { return cantidad; }

```

```

    public void setCantidad(int cantidad) { this.cantidad =
cantidad; }

    @Override
    public String toString() {
        return "ComandaPlato{comanda=" + (comanda != null ?
comanda.getId() : null) +
            ", plato=" + (plato != null ? plato.getNombre() :
null) + ", cantidad=" + cantidad + "}";
    }
}

```

modelo/ComandaPlatoId.java:

```

package modelo;

import jakarta.persistence.*;
import java.io.Serializable;
import java.util.Objects;

@Embeddable
public class ComandaPlatoId implements Serializable {
    private Long comandaId;
    private Long platoId;

    public ComandaPlatoId() {}
    public ComandaPlatoId(Long comandaId, Long platoId) {
this.comandaId = comandaId; this.platoId = platoId; }

    // getters / setters
    public Long getComandaId() { return comandaId; }
    public void setComandaId(Long comandaId) { this.comandaId =
comandaId; }
    public Long getPlatoId() { return platoId; }
    public void setPlatoId(Long platoId) { this.platoId = platoId;
}

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof ComandaPlatoId)) return false;
        ComandaPlatoId that = (ComandaPlatoId) o;
        return Objects.equals(comandaId, that.comandaId) &&
Objects.equals(platoId, that.platoId);
    }

    @Override

```

```
public int hashCode() { return Objects.hash(comandaId,
platoId); }
}
```

dao/ClienteDAO.java:

```
package dao;

import modelo.Cliente;
import modelo.Comanda;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.time.LocalDateTime;
import java.util.List;

public class ClienteDAO {

    public void guardar(Cliente c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Cliente obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Cliente.class, id);
        }
    }

    public List<Cliente> listarTodos() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Cliente",
Cliente.class).list();
        }
    }
}
```



```

    }
}

    public void actualizar(Cliente c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Cliente c = s.find(Cliente.class, id);
            if (c == null) return;

            tx = s.beginTransaction();
            s.remove(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Comanda> listarComandasDeCliente(Long idCliente) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            String hql = "FROM Comanda c WHERE c.cliente.id =
:idCli";
            return s.createQuery(hql, Comanda.class)
                .setParameter("idCli", idCliente)
                .list();
        }
    }
}

```

dao/CamareroDAO.java:

```
package dao;

import modelo.Camarero;
import modelo.Comanda;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.time.LocalDateTime;
import java.util.List;

public class CamareroDAO {

    public void guardar(Camarero c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Camarero obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Camarero.class, id);
        }
    }

    public List<Camarero> listarTodos() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Camarero",
Camarero.class).list();
        }
    }
}
```

```

    public void actualizar(Camarero c) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Camarero c = s.find(Camarero.class, id);
            if (c == null) return;

            tx = s.beginTransaction();
            s.remove(c);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Comanda> listarComandasDeCamarero(Long idCamarero)
    {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            String hql = "FROM Comanda c WHERE c.camarero.id =
:idCam";
            return s.createQuery(hql, Comanda.class)
                .setParameter("idCam", idCamarero)
                .list();
        }
    }
}

```

dao/MesaDAO.java:

```
package dao;

import modelo.Mesa;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class MesaDAO {

    public void guardar(Mesa m) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(m);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Mesa obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Mesa.class, id);
        }
    }

    public List<Mesa> listarTodas() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Mesa", Mesa.class).list();
        }
    }
}
```

```

    public void actualizar(Mesa m) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(m);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Mesa m = s.find(Mesa.class, id);
            if (m == null) return;

            tx = s.beginTransaction();
            s.remove(m);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void actualizarEstado(Long idMesa, String estado) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            Mesa m = s.find(Mesa.class, idMesa);
            if (m != null) {
                m.setEstado(estado);
                s.merge(m);
            }
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }
}

```

```
// Listar mesas ocupadas con número de comandas activas
public List<Object[]> mesasOcupadasConCuentaComandas() {
    try (var s =
HibernateUtil.getSessionFactory().openSession()) {
        String hql =
            "SELECT m, COUNT(c) " +
            "FROM Mesa m JOIN m.comandas c " +
            "WHERE m.estado = 'ocupada' " +
            "GROUP BY m";
        return s.createQuery(hql, Object[].class).list();
    }
}
}
```

dao/PlatoDAO.java:

```
package dao;

import modelo.Plato;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class PlatoDAO {

    public void guardar(Plato p) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.persist(p);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public Plato obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Plato.class, id);
        }
    }
}
```

```

    }

    public List<Plato> listarTodos() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Plato", Plato.class).list();
        }
    }

    public void actualizar(Plato p) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(p);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminar(Long id) {
        Transaction tx = null;
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            Plato p = s.find(Plato.class, id);
            if (p == null) return;

            tx = s.beginTransaction();
            s.remove(p);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Plato> listarPorCategoria(String categoria) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Plato p WHERE p.categoria =
:cat", Plato.class)
                .setParameter("cat", categoria)

```

```

        .list();
    }
}
}

```

dao/ComandaDAO.java:

```

package dao;

import modelo.Comanda;
import modelo.ComandaPlato;
import modelo.Plato;
import util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.time.LocalDateTime;
import java.util.List;

public class ComandaDAO {

    public Comanda crearComanda(Comanda comanda) {
        Transaction tx = null;

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();

            // Asegurar que cada item conoce la comanda
            for (ComandaPlato cp : comanda.getItems()) {
                cp.setComanda(comanda);
            }

            s.persist(comanda);
            tx.commit();
            return comanda;

        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }

        return null;
    }
}

```



```

    }

    public Comanda obtener(Long id) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.find(Comanda.class, id);
        }
    }

    public List<Comanda> listarTodas() {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            return s.createQuery("FROM Comanda",
Comanda.class).list();
        }
    }

    public void actualizar(Comanda comanda) {
        Transaction tx = null;

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();
            s.merge(comanda);
            tx.commit();

        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public void eliminarComanda(Long idComanda) {
        Transaction tx = null;

```

```

        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            tx = s.beginTransaction();

            Comanda c = s.find(Comanda.class, idComanda);
            if (c != null) s.remove(c);

            tx.commit();

        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    // Consultas HQL

    // Listar comandas de una mesa concreta
    public List<Comanda> comandasPorMesa(Long idMesa) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            String hql = """
            SELECT DISTINCT c
            FROM Comanda c
            LEFT JOIN FETCH c.items cp
            LEFT JOIN FETCH cp.plato
            WHERE c.mesa.id = :idMesa
            """;

            return s.createQuery(hql, Comanda.class)
                .setParameter("idMesa", idMesa)
                .list();
        }
    }

    // Consultar comandas de un cliente mostrando mesa y camarero
    public List<Comanda> comandasPorCliente(Long idCliente) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {
            String hql = "FROM Comanda c WHERE c.cliente.id =
:idCli";

            return s.createQuery(hql, Comanda.class)
                .setParameter("idCli", idCliente)
                .list();
        }
    }

```

```

    }

    // Total facturado por un camarero en un rango de fechas
    public Double totalFacturadoPorCamarero(Long idCamarero,
LocalDateTime desde, LocalDateTime hasta) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {

            String hql = """
                SELECT SUM(item.cantidad * p.precio)
                FROM Comanda c
                JOIN c.items item
                JOIN item.plato p
                WHERE c.camarero.id = :idCam
                AND c.fechaHora BETWEEN :ini AND :fin
            """;

            Double total = s.createQuery(hql, Double.class)
                .setParameter("idCam", idCamarero)
                .setParameter("ini", desde)
                .setParameter("fin", hasta)
                .uniqueResult();

            return total != null ? total : 0.0;
        }
    }

    // Total facturado por un cliente en un rango de fechas
    public Double totalFacturadoPorCliente(Long idCliente,
LocalDateTime desde, LocalDateTime hasta) {
        try (Session s =
HibernateUtil.getSessionFactory().openSession()) {

            String hql = """
                SELECT SUM(item.cantidad * p.precio)
                FROM Comanda c
                JOIN c.items item
                JOIN item.plato p
                WHERE c.cliente.id = :idCli
                AND c.fechaHora BETWEEN :ini AND :fin
            """;

            Double total = s.createQuery(hql, Double.class)
                .setParameter("idCli", idCliente)
                .setParameter("ini", desde)
                .setParameter("fin", hasta)
                .uniqueResult();

```

```

        return total != null ? total : 0.0;
    }
}

// Total facturado por categoría de plato
public List<Object[]> totalFacturadoPorCategoria() {
    try (Session s =
HibernateUtil.getSessionFactory().openSession()) {

        String hql = ""
        SELECT p.categoria, SUM(cp.cantidad * p.precio)
        FROM Comanda c
        JOIN c.items cp
        JOIN cp.plato p
        GROUP BY p.categoria
        """;

        return s.createQuery(hql, Object[].class).list();
    }
}
}

```

util/HibernateUtil.java:

```

package util;

import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;

public class HibernateUtil {
    private static final SessionFactory sessionFactory =
buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            StandardServiceRegistry registry = new
StandardServiceRegistryBuilder()
                .configure() // busca hibernate.cfg.xml en
resources
                .build();
            Metadata metadata = new
MetadataSources(registry).getMetadataBuilder().build();
            return metadata.getSessionFactoryBuilder().build();
        } catch (Exception e) {

```

```

        System.err.println("Initial SessionFactory creation
failed: " + e);
        throw new ExceptionInInitializerError(e);
    }
}

    public static SessionFactory getSessionFactory() { return
sessionFactory; }
    public static void shutdown() { getSessionFactory().close(); }
}

```

hibernate.cfg.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.connection.driver_class">org.sqlite.JDBC</property
>
        <property
name="hibernate.connection.url">jdbc:sqlite:restaurante.db</proper
ty>
        <property
name="hibernate.dialect">org.hibernate.community.dialect.SQLiteDia
lect</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="modelo.Mesa"/>
        <mapping class="modelo.Plato"/>
        <mapping class="modelo.Camarero"/>
        <mapping class="modelo.Cliente"/>
        <mapping class="modelo.Comanda"/>
        <mapping class="modelo.ComandaPlato"/>
    </session-factory>
</hibernate-configuration>

```

pom.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>act3ut3</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>26</maven.compiler.source>
        <maven.compiler.target>26</maven.compiler.target>
</project>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <hibernate.version>7.1.0.Final</hibernate.version>
    <sqlite.jdbc.version>3.50.3.0</sqlite.jdbc.version>
    <slf4j.version>2.0.17</slf4j.version>
</properties>

    <dependencies>
        <!-- Hibernate core (incluye implementación JPA) -->
        <dependency>
            <groupId>org.hibernate.orm</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>${hibernate.version}</version>
        </dependency>

        <!-- Community Dialects (SQLite) -->
        <dependency>
            <groupId>org.hibernate.orm</groupId>
            <artifactId>hibernate-community-dialects</artifactId>
            <version>${hibernate.version}</version>
        </dependency>

        <!-- SQLite JDBC -->
        <dependency>
            <groupId>org.xerial</groupId>
            <artifactId>sqlite-jdbc</artifactId>

```

```
        <version>${sqlite.jdbc.version}</version>
    </dependency>

    <!-- SLF4J Simple -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>${slf4j.version}</version>
    </dependency>
</dependencies>

</project>
```

Acceso al zip del proyecto

<https://github.com/RafaelMayor/AED/tree/main/act3ut3>